

A simple proposal for unifying generic and object-oriented programming

In this paper, we propose a simple extension to [P1141R1](#) that we believe provides a simple and comprehensive proposal for unifying generic and object-oriented programming.

The problem

C++ works best when compile-time and run-time programming are similar. For example, constexpr functions allow programs to easily move calculations back and forth between compile-time and run-time as circumstances dictate with no change in code. Likewise, it is well-known that generic programming and object-oriented programming can be applied to similar problems (e.g., Bjarne Stroustrup's [Concepts for C++1y: The Challenge](#)), but they do so with very different notations. Once the design decision is made about whether to use templates or virtual functions, it is extremely committal and cannot easily be changed.

For example, in designing a drawing library, one would need to decide whether to provide distinct shapes via a Shape base class providing runtime dispatch with virtual functions or a Shape concept providing compile-time dispatch. If Shape is a class, a function might be declared as

```
bool is_convex(Shape const &s);
```

However, if Shape is a concept, a very different declaration must be used:

```
template<typename T> requires Shape<T> bool is_convex(T const &s);
```

Once this choice is made, the programmer is fully committed to one of two very different programming paradigms with complex tradeoffs from day one.

Terse notation, as proposed most recently in P1141R1, reduces the notational gap between object-oriented and template programming:

```
bool is_convex(Shape auto const &s);
```

However, switching Shape between a class and a concept still requires changing the declaration of is_convex, and even if the declaration were the same (as in the [Concepts TS](#)), many changes would undoubtedly need to be made to the function body to accommodate changing Shape from a class to a concept. E.g.,

```
// Body is ok if Shape is a class but badly broken if Shape is a concept
bool is_convex(Shape const &s)
{
    pair<Shape, int> p(s, 7);
    auto x = make_unique<optional<Shape>>(convex_hull(s));
    /* ... */
}
```

In the end, the obvious similarity between compile-time and runtime dispatch sometimes feels more like something that is there to tantalize and frustrate us than something that can be effectively leveraged.

Our Solution

So, how do we create code that works equally well regardless of whether Shape is a class or a concept? Fortunately, P1141R1 provides auto as a sigil that indicates something that looks like a type could be a concept.

```
// is_convex is a function template
bool is_convex(Shape auto const &s) { /* ... */ }
```

P1141R1 defines what this means when Shape is a concept. If we want this code to work regardless of whether Shape is a concept or a class, we need to define what it means when Shape is a class:

First, we introduce one notation. If C is a class, we let `inherits_from_C<T>` denote the concept asserting that T publicly inherits from C . For example, if `Shape` is a class, then `inherits_from_Shape` is the concept

```
template<typename T> concept bool inherits_from_Shape = is_convertible_v<T *, Shape *>;
```

Now we can state our proposal simply as:

Proposal

With the above notation, we propose everything as in P1141R1 with the additional rules that if C is a class,

- Occurrences of `C auto` are replaced by `inherits_from_C auto`.
- Implicit conversions
- If a function template has a `C auto` in its parameter list, any occurrences of C within the function body are also replaced by `inherits_from_C auto`

That's it! Let's make it concrete illustrative examples

Illustrative examples

As noted above, P1141R1 explains what

```
bool is_convex(Shape auto const &s);
```

means if `Shape` is a concept. What we propose is that if `Shape` is a class, then the above is rewritten as

```
bool is_convex(inherits_from_Shape auto const &s);
```

In this way, we ensure that the declaration of `is_convex` makes sense regardless of whether `Shape` is a class or a concept. Furthermore, since all occurrences of `Shape` within the body of `is_convex` are also replaced by the `inherits_from_Shape` concept, all of the subtle distinctions between classes and concepts are mooted, and we don't need to worry that we will create brittle code that inadvertently assumes that `Shape` is a class.

We illustrate this ease of writing flexible function bodies with another example

```
// Want to work identically whether Shape is a concept or type
bool sameShapeMaybeDifferentPosition(Shape auto const &s1, Shape auto const &s2) {
    Shape const &s1Translated = s1 - s1.lower_left();
    Shape const &s2Translated = s2 - s2.lower_left();
    return sameShape(s1Translated, s2Translated);
}
```

To see that this code works the same way whether `Shape` is a class or concept, let us examine both cases

Shape is a concept

In this case, it means exactly what it does in P1141R1: A concept-constrained function template. We note particular that, by independent binding, `s1` and `s2` may have different types, such as `triangle` and `ellipse`.

Shape is a class

On the other hand, if `Shape` is now a class with virtual functions that are implemented by derived classes `Triangle` and `Ellipse`, the above function still works without changes. In this case, the compiler reinterprets it as

```
// Want to work identically whether Shape is a concept or type
bool sameShapeMaybeDifferentPosition(inherits_from_Shape auto const &s1, inherits_from_Shape auto const &s2) {
    inherits_from_Shape const &s1Translated = s1 - s1.lower_left();
    inherits_from_Shape const &s2Translated = s2 - s2.lower_left();
    return sameShape(s1Translated, s2Translated);
}
```

so if it is called, for example, with two `Shape &` arguments, instead of doing compile-time dispatch as it did with the above example, it does runtime dispatch with virtual function dispatch. We note again that because the objects' runtime types may differ, that this correctly corresponds to the independent binding of the concepts version above.

Changing Shape between a class and a concept

As we see from the above example, Shape can be changed back and forth between a class and a concept during tuning and as needs evolve in the future, replacing a brittle, premature, and committal decision with robust and flexible code that works equally well with object-orientation and generics.

The role of inferencing

Looking at the body for `is_convex` at the top of this paper, we see that the code implicitly assumes that Shape is a class in many places. While the above proposal ensures that classes that may be replaced with concepts later will not be able to use such code, that may be perceived as too high a price if programming with concepts is much more difficult than programming with classes. Indeed, irrespective of this proposal, Bjarne Stroustrup and others have likewise noted that for concepts to reach its full potential, code written with concepts need not be much harder than programming with types and that safe and easy constrained inferencing is key to that.

So how should the body of `is_convex` have been written? As the reader might guess, constrained inferencing as provided by P1141R1 and robust class template argument deduction are the key.

```
// The following is now ok regardless of whether Shape is a class or a concept
bool is_convex(Shape auto const &s)
{
    pair p(s, 7);
    auto x = make_unique<optional>(convex_hull(s)); // Assumes P1069R0
    /* ... */
}
```

In fact, we would go so far as to say that the code is clearer, more robust, and flexible than the original class-based code (We understand that contrary examples can be constructed, but that does not detract from the value of writing code that is agnostic between classes and concepts).

Indeed, we would go even further to say that constrained inferencing as provided by concepts and class template argument deduction is key to writing flexible scalable code, one of the most important problems in programming. As has long been known, unconstrained generic code is very flexible but is difficult to scale as millions of lines of unconstrained types will leave the programmer thoroughly perplexed (this is of course one of the main problems concepts was designed to solve). On the other hand, code written with explicit types is clear, but also fails to scale as it does not separate an object's implementation from the way it is used, resulting in tight coupling where changing the type of an object to a different class with the same external behavior can require thousands of changes scattered through the code. As a result, large programs become brittle and difficult to evolve over time and successive releases can no longer provide rapid improvement (or often, any meaningful improvement at all).

We believe that with safe and easy constrained inferencing, loose coupling can be maintained, and writing code to work for both classes and concepts as described in this paper will result in it being enhanced rather than compromised. See [P1168R0](#) for a more detailed look with concrete examples at how robust constrained inferencing terse notation and class template argument deduction can enable flexible, readable, and scalable code.

Conclusion

By building on P1141R1 to allow *classname* auto according to the above rules, writing code that flexibly works with both classes and concepts becomes as easy as writing concepts code alone, providing a simple unification between the two major programming styles in C++, without the need to get mired in the myriad subtle differences between generic and object-oriented programming.